Lecture 4:

SQL: The Structured Query Language

Topic Objectives

- Overview of The SQL Query Language
- Learn basic SQL statements for creating database structures
- Learn SQL statements to add data to a database
- Learn basic SQL SELECT statements and option for querying a single table
- Learn basic SQL SELECT statements for querying multiple tables with subqueries
- Learn basic SQL SELECT statements for querying multiple tables with joins
- Learn SQL statements to modify and delete data from a database
- Learn SQL statements to modify and delete database tables and constraints

History

- IBM Sequel language developed as part of System R project at the IBM San Jose Research Laboratory
- Renamed Structured Query Language (SQL)
- ANSI and ISO standard SQL:
  - SQL-86
  - SQL-89
  - SQL-92
  - SQL:1999 (language name became Y2K compliant!)
  - SQL:2003
- Commercial systems offer most, if not all, SQL-92 features, plus varying feature sets from later standards and special proprietary features.
  - Not all examples here may work on your particular system.

Structured Query Language

- Structured Query Language
  - Acronym: SQL
  - Pronounced "*Sequel*" or "*S-Q-L*"
  - Originally developed by IBM as the SEQUEL language in the 1970s
  - Designed to support Edgar Codd's relational model
    - Codd, E.F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6): 377-387
  - Based on relational algebra
  - ANSI / ISO standard


Microsoft
SQL Server
Linux
IBM
DB2
Microsoft
Windows
android
UNIX
MySQL
iOS
Mac
ORACLE

SQL Defined

- SQL is not a programming language, but rather is a data sub-language
- SQL is composed of :
  - *A data-definition language (DDL)*

  Used to define and manage database structure

  - *A data manipulation language (DML)*

  Data definition and updating

  Data retrieval (Queries)

  -A data Control language (DCL)

  For creating user accounts, managing permission

SQL for Data Definition

- The SQL data definition statements include
  - CREATE

    To create database objects
  - ALTER

    To modify the structure and/or characteristics of existing database objects
  - DROP

    To delete existing database objects

Domain Types in SQL

- **char(n).** Fixed length character string, with user-specified length *n*.
- **varchar(n).** Variable length character strings, with user-specified maximum length *n*.
- **int.** Integer (a finite subset of the integers that is machine-dependent).
- **smallint.** Small integer (a machine-dependent subset of the integer domain type).
- **numeric(p,d).** Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (ex., **numeric**(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)
- **real, double precision.** Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n).** Floating point number, with user-specified precision of at least *n* digits.

Create Table Construct

- An SQL relation is defined using the **create table** command:

  create table r (A_1 D_1, A_2 D_2, …, A_n D_n,
  (integrity-constraint$_1$),
  …,
  (integrity-constraint$_k$))

  - r is the name of the relation
  - each A_i is an attribute name in the schema of relation r
  - D_i is the data type of values in the domain of attribute A_i
- Example:

  create table *instructor* (
  ID **char**(5),
  name **varchar**(20),
  dept_name **varchar**(20),
  salary **numeric**(8,2))

SQL for Data Definition: CREATE

- Creating database table
- The SQL CREATE TABLE statement

```
create table Employee (
 empId          Integer    NOT NULL,
empName         char(25)   NOT NULL
   );
```

SQL for Data Definition: CREATE with CONSTRAINT 1

- Creating database tables with PRIMARY KEY constraints
  - The SQL CREATE TABLE statement
  - The SQL CONSTRAINT keyword

```
CREATE TABLE Employee (
    empId        Integer      NOT NULL,
    empName      Char(25)     NOT NULL,
    CONSTRAINT   empPk        PRIMARY KEY(empId)
);
```

SQL for Data Definition: CREATE with CONSTRAINT -3

- Creating database table using PRIMARY KEY and FOREIGN KEY constraints
- The SQL CREATE TABLE statement
- The SQL CONSTRAINT keyword

```
create table EmployeeSkill (
 empId          Integer     NOT NULL,
skillId         Integer     NOT NULL,
skillLevel          Integer          NULL,
CONSTRAINT          empskillPK    PRIMARY KEY (empId, skillId),
CONSTRAINT          empFK         FOREIGN KEY (empId)
   REFERENCES Employee (empId) ,
CONSTRAINT          skillFK       FOREIGN KEY (skillId)
   REFERENCES Employee (skillId) ,
    );
```

Employee

empId	empName
1	Dan
2	Penny
3	Sheldon

EmployeeSkill

empId	skillId	skillLevel
1	101	5
1	102	3
2	101	4
2	103	5
3	102	3
3	104	5

Skill

skillId	skillName
101	Sleeping
102	Watching TV
103	Waiting tables
104	M-theory

SQL for Data Definition: CREATE with CONSTRAINT -4

- Creating database table using PRIMARY KEY and FOREIGN KEY constraints
- The SQL CREATE TABLE statement
- The SQL CONSTRAINT keyword
- ON UPDATE CASCADE and on DELETE CASCADE

```
create table EmployeeSkill (
 empId          Integer       NOT NULL,
skillId         Integer       NOT NULL,
skillLevel      Integer       NULL,
CONSTRAINT      empskillPK    PRIMARY KEY (empId, skillId),
CONSTRAINT      empFK         FOREIGN KEY (empId)
   REFERENCES Employee (empId) ON DELETE CASCADE,
CONSTAINT  skillFK            FOREIGN KEY (skillId)  REFERENCES
   Employee (skillId) ON UPDATE CASCADE
    );
```

PRIMARY KEY: ALTER 1

- Adding primary key constraints to an existing table
- The SQL ALTER statement

ALTER TABLE *Employee*

ADD CONSTRAINT empPK PRIMARY KEY (empId);

PRIMARY KEY: ALTER 2

- Adding a composite primary key constraints to an existing table
- The SQL ALTER statement

ALTER TABLE *EmployeeSkill*

ADD CONSTRAINT empSkillPK PRIMARY KEY (empId, skillId);

PRIMARY KEY: ALTER 3

- Adding a foreign key constraints to an existing table
- The SQL ALTER statement

ALTER TABLE *Employee*

ADD CONSTRAINT empFK FOREIGN KEY (deptId) REFERENCES Department (deptId) ;

Modifying Data Using SQL

- INSERT INTO

Will add a new row to a table

- UPDATE

Will update the rows in a table which match the specified criteria

- DELETE FROM

Will delete the rows in a table which match the specified criteria

Adding Data: INSERT INTO

- To add a row to an existing table, use the INSERT INTO statement
- Non-numeric data must be enclosed in single quotes (')

```
INSERT INTO Employee (empId, salaryCode, lastName)
    VALUES (62, 11, 'Halpert');
```

Modifying Data using SQL:
Changing Data Values: UPDATE

- To change the data values in an existing row (or a set of rows) use the UPDATE statement

```
UPDATE    Employee
SET       phone = '657-278-1234'
WHERE     empId = 29;

UPDATE    Employee
SET       deptId = 4
WHERE     empName LIKE 'Da%';

UPDATE    Employee
SET       deptId = 3;
```

Modifying Data using SQL:
Deleting Data: DELETE

- To delete a row (or a set of rows) from a table use the DELETE FROM statement

```
DELETE FROM Employee
WHERE  empId = 29;

DELETE FROM Employee
WHERE  empName LIKE 'Da%';

DELETE FROM Employee;
```

Modifying Data using SQL:
Changing Data Values: UPDATE

- To change the data values in an existing row (or a set of rows) use the UPDATE statement

```
UPDATE    Employee
SET       phone = '657-278-1234'
WHERE     empId = 29;

UPDATE    Employee
SET       deptId = 4
WHERE     empName LIKE 'Da%';

UPDATE    Employee
SET       deptId = 3;
```

Modifying Data using SQL:
Deleting Data: DELETE

- To delete a row (or a set of rows) from a table use the DELETE FROM statement

```
DELETE FROM Employee
WHERE  empId = 29;

DELETE FROM Employee
WHERE  empName LIKE 'Da%';

DELETE FROM Employee;
```

SQL for Data Retrieval: Queries

- SELECT is the best known SQL statement
- SELECT will retrieve information from the database that matches the specified criteria using a SELECT/FROM/WHERE framework

```
SELECT      empName
FROM        Employee
WHERE       empId = 33;

SELECT      empName
FROM        Employee;
```

SQL for Data Retrieval: The Resultset is a Relation

- A query pulls information from one or more relations and creates (temporarily) a new relation
- This allows a query to:
  - Create a new relation
  - Feed information into another query (as a *subquery*)
  - The resultset may not be in 3NF
    - Especially when performing a *join*

SQL for Data Retrieval: Displaying Multiple Columns

- To show values for two or more specific columns, use a comma-separated list of column names

```
SELECT empId, empName
FROM   Employee;
```

SQL for Data Retrieval: Displaying All Columns

- To show all of the column values for the rows that match the specified criteria, use an asterisk (*)

```
SELECT *
FROM   Employee;
```

SQL for Data Retrieval:
Showing Each Row Only Once

- The DISTINCT keyword may be added to the SELECT statement to suppress the display of duplicate rows

```
SELECT DISTINCT deptId
FROM   Employee;
```

SQL for Data Retrieval:
Specifying Search Criteria

- The WHERE clause specifies the matching or filtering criteria for the records (rows) that are to be displayed

```
SELECT    empName
FROM      Employee
WHERE     deptId = 15;
```

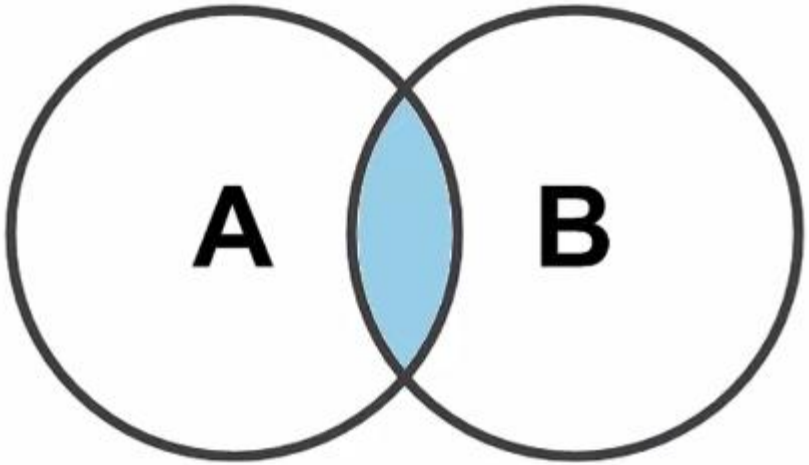
SQL for Data Retrieval: Match Criteria

- WHERE clause comparisons may include
  - Equals "="
  - Not Equals "<>" or "!="
  - Greater than ">"
  - Less than "<"
  - Greater than or Equal to ">="
  - Less than or Equal to "<="

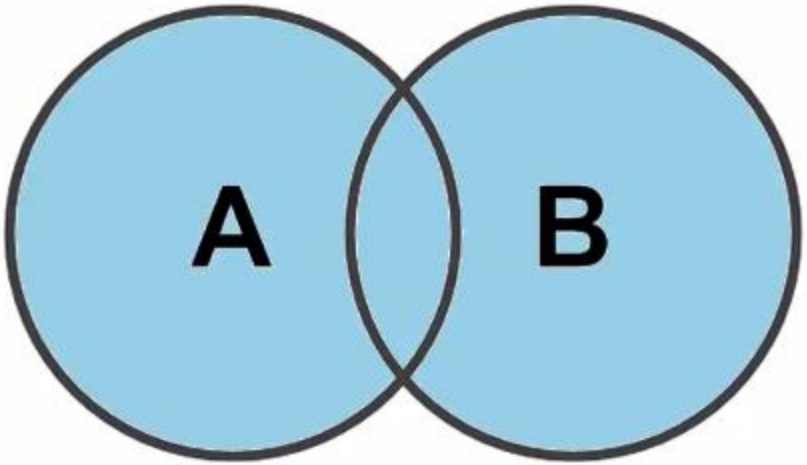
SQL for Data Retrieval:
Match Operators

- Multiple matching criteria may be specified using
  - AND
    - Representing an intersection of the data sets
  - OR
    - Representing a union of the data sets
  - Concepts such as "intersection" and "union" are derived from relational algebra
    - Venn diagrams!

AND operation



OR operation



SQL for Data Retrieval: Operator Examples

```
SELECT  empName
FROM    Employee
WHERE   deptId < 7 OR deptId > 12;
```

```
SELECT empName
FROM   Employee
WHERE  deptId = 9 AND salaryCode <= 3;
```

SQL for Data Retrieval: A List of Values

- The WHERE clause may include the IN keyword to specify that a particular column value must match one of the values in a list
  - This is much more convenient than using a series of "OR" operators!

```
SELECT    empName
FROM      Employee
WHERE     deptId IN (4, 8, 9);
```

  - Compare to:

```
WHERE     deptId = 4 OR deptId = 8 OR deptId = 9;
```

SQL for Data Retrieval: The Logical NOT Operator

- Any criteria statements can be preceded by a NOT operator in order to invert the results
  - Using NOT will return all information *except* the information matching the specified criteria

```
SELECT    empName
FROM      Employee
WHERE     deptId NOT IN (4, 8, 9);
```

SQL for Data Retrieval: Finding Data in a Range of Values

- SQL provides a BETWEEN keyword that allows a user to specify a minimum and maximum value on one line
  - BETWEEN is inclusive!!

```
SELECT   empName
FROM     Employee
WHERE    salaryCode BETWEEN 10 AND 45;
```

  - Compare to:

```
WHERE    salaryCode >= 10 AND salaryCode <= 45;
```

SQL for Data Retrieval: Wildcard Searches

- The SQL LIKE keyword allows for searches on partial data values
- LIKE can be paired with wildcards to find rows that partially match a string value
  - The multiple character wildcard character is a percent sign (%)
  - The single character wildcard character is an underscore (_)

SQL for Data Retrieval: Wildcard Search Examples

```
SELECT      empId
FROM        Employee
WHERE       empName LIKE 'Da%';

SELECT      empId
FROM        Employee
WHERE       phone LIKE '657-278-____';
```

SQL for Data Retrieval: Sorting the Resultset

- Query results may be sorted using the ORDER BY clause
  - Ascending vs. descending sorts

```
SELECT      *
FROM        Employee
ORDER BY    empName;
```

SQL for Data Retrieval:
Built-in Numeric Functions

- SQL provides several built-in numeric functions
  - COUNT
    - Counts the number of rows that match the specified criteria
  - MIN
    - Finds the minimum value for a specific column for those rows matching the criteria
  - MAX
    - Finds the maximum value for a specific column for those rows matching the criteria

SQL for Data Retrieval: Built-in SQL Functions (continued)

- SUM
  - Calculates the sum (total) for a specific column for those rows matching the criteria
- AVG
  - Calculates the numerical average (mean) of a specific column for those rows matching the criteria
- STDEV
  - Calculates the standard deviation of the values in a numeric column whose rows match the criteria

SQL for Data Retrieval: Built-in Function Examples

```
SELECT COUNT(*)
FROM   Employee;


SELECT MIN(hours) AS minimumHours,
       MAX(hours) AS maximumHours,
       AVG(hours) AS averageHours
FROM   Project
WHERE  ProjID > 7;
```

SQL for Data Retrieval:
Providing Subtotals: GROUP BY

- Categorized results can be retreived using the GROUP BY clause

```
SELECT      deptId,
            COUNT(*) AS numberOfEmployees
FROM        Employee
GROUP BY    deptId;
```

deptId	numOfEmployees
1	23
2	47
3	18
4	21

SQL for Data Retrieval:
Providing Subtotals: GROUP BY with HAVING

- The HAVING clause may optionally be used with a GROUP BY in order to restrict which categories are displayed

```
SELECT      salespersonId, salespersonLastName,
            SUM(saleAmount) AS totalSales
FROM        Sales
GROUP BY    salespersonId, salespersonLastName
HAVING      SUM(saleAmount) >= 10000;
```

SQL for Data Retrieval:
Retrieving Information from Multiple Tables

- Subqueries
  - As stated earlier, the result of a query is a *relation*. The results from one query may therefore be used as input for another query. This is called a *subquery.*
    - Noncorrelated subqueries
    - Correlated subqueries

SQL for Data Retrieval: Noncorrelated Subquery Example

- In a noncorrelated subquery, the inner query only needs to run once in order for the database engine to solve the problem

```
SELECT empName
FROM   Employee
WHERE  deptId IN
       (SELECT deptId
        FROM   Department
        WHERE  deptName LIKE 'Account%');
```

SQL for Data Retrieval: Correlated Subquery Example

- In a correlated subquery, the inner query needs to be run repeatedly in order for the database engine to solve the problem.
  - The inner query needs a value from the outer query in order to run.

```
SELECT empName
FROM   Employee e
WHERE  empSalary >
       (SELECT AVG(empSalary)
        FROM   Employee
        WHERE  deptId = e.deptId);
```

Employee

empId	empName	empSalary	deptId
1	Dan	20000	5
2	Raj	50000	2
3	Sheldon	110000	5
4	Penny	90000	3
5	Bernadette	120000	5